

```
# Import de l'objet db qui contient la configuration
SQLAlchemy
from database import db

# Import des outils SQLAlchemy 2.x pour définir les types
et colonnes des modèles
from sqlalchemy.orm import Mapped, mapped_column,
relationship

# Import des types de colonnes SQL (texte, entier...)
from sqlalchemy import String, Integer

Afficher plus ▾
```

Ton modèle est globalement correct, mais il y a quelques points à améliorer :

1. `from_body()` **n'est pas idéal dans un modèle SQLAlchemy** : ce nom peut prêter à confusion car ce n'est pas une méthode de création depuis un dictionnaire mais plutôt une méthode de mise à jour. Je te propose de la renommer `update_from_body()` (mais tu peux garder le nom si ton projet l'exige).
2. `price = body["price"] * 100` peut créer un problème si `price` arrive en float (15.99) à cause des arrondis. Il vaut mieux utiliser `int(round(...))` .
3. `from participants import Participant` peut provoquer une importation circulaire si `Participant` importe aussi `Course` . Une solution fréquente est d'utiliser `TYPE_CHECKING` .
4. Ajout de commentaires plus propres et cohérents.

Voici une version corrigée et commentée :

Python



▶ Exécuter

```
# Import de l'objet db contenant la configuration SQLAlchemy
from database import db

# Import des outils SQLAlchemy 2.x
# Mapped permet de typer les attributs du modèle
# mapped_column permet de définir les colonnes SQL
# relationship permet de créer des relations entre les tables
from sqlalchemy.orm import Mapped, mapped_column, relationship

# Import des types de colonnes SQL
from sqlalchemy import String, Integer
```

```
# Utilisé uniquement pour les annotations de type
# Cela évite les problèmes d'import circulaire
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from participants import Participant

# Modèle Course représentant la table "courses"
class Course(db.Model):

    # Nom de la table dans la base de données
    __tablename__ = "courses"

    # Identifiant unique du cours
    # primary_key=True indique que cette colonne est la clé primaire
    id: Mapped[int] = mapped_column(primary_key=True)

    # Nom du cours
    # Obligatoire et limité à 100 caractères
    name: Mapped[str] = mapped_column(
        String(100),
        nullable=False
    )

    # Nom du coach responsable du cours
    # Obligatoire et limité à 100 caractères
    coach_name: Mapped[str] = mapped_column(
        String(100),
        nullable=False
    )

    # Nombre maximum de participants autorisés
    seats: Mapped[int] = mapped_column(
        Integer,
        nullable=False
    )

    # Prix du cours stocké en centimes
    #
    # Exemple :
    # 1500 dans la base = 15,00 €
    #
    # Stocker en centimes évite les erreurs liées aux nombres flottants
    price: Mapped[int] = mapped_column(
```

```

        Integer,
        nullable=False
    )

    # Relation entre un cours et ses participants
    #
    # Un cours peut avoir plusieurs participants.
    #
    # Exemple :
    # course.participants
    # retourne la liste des participants inscrits.
    participants: Mapped[list["Participant"]] = relationship(
        back_populates="course",

        # Si un cours est supprimé,
        # ses participants associés sont supprimés automatiquement.
        cascade="all, delete-orphan"
    )

    # Propriété calculée indiquant le nombre de places restantes
    #
    # Exemple :
    # course.remaining_seats
    #
    # Si le cours possède 20 places et 5 participants :
    # remaining_seats = 15
    @property
    def remaining_seats(self) -> int:

        return self.seats - len(self.participants)

    # Transformation de l'objet Course en dictionnaire Python
    #
    # Cette méthode est pratique pour retourner les données
    # sous forme JSON dans une API Flask.
    def to_dict(self) -> dict:

        return {

            # Identifiant du cours
            "id": self.id,

            # Nom du cours
            "name": self.name,

            # Nom du coach
            "coach_name": self.coach_name,

```

```
# Nombre total de places
"seats": self.seats,

# Conversion des centimes vers les euros
#
# Exemple :
# 1500 devient 15.0
"price": self.price / 100,

# Nombre de places disponibles restantes
"remaining_seats": self.remaining_seats
}

# Mise à jour d'un cours depuis les données reçues
# généralement depuis un body JSON d'une requête API.
#
# Exemple de body :
#
# {
#   "name": "Yoga",
#   "coach_name": "Marie",
#   "seats": 20,
#   "price": 15
# }
def update_from_body(self, body: dict):

    # Mise à jour du nom du cours
    self.name = body["name"]

    # Mise à jour du nom du coach
    self.coach_name = body["coach_name"]

    # Mise à jour du nombre de places
    self.seats = body["seats"]

    # Conversion du prix en centimes
    #
    # Exemple :
    # 15.99 € devient 1599 centimes
    self.price = int(round(body["price"] * 100))

    # Retourne l'objet modifié
    # Permet d'enchaîner les opérations
    return self
```

Modifications principales :

- ✓ Commentaires corrigés et plus explicites
- ✓ Gestion du risque d'import circulaire
- ✓ Conversion du prix sécurisée avec `round()`
- ✓ Méthode `from_body()` renommée en `update_from_body()`
- ✓ Code compatible SQLAlchemy 2.x

Ton modèle est maintenant propre pour une API Flask + SQLAlchemy.

